Faculty of Engineering
Ain Shams University

CSE 488 — Deep Learning

Class Project Report

Conversational Professor Digital Twin: Voice Cloning Pipeline with Real-Time ASR and TTS

End-to-end preprocessing, LoRA fine-tuning, and WebSocket deployment

Team Members

Ahmed Farrag (23P0168)
Ahmed Nasri (23P0270)

Under the supervision of

Dr. Alaa Hamdy
Eng. Karim Bassel

May 2026

Abstract

We present an end-to-end pipeline for building *voice digital twins* of multiple university professors from lecture recordings. Raw long-form audio is converted into training-ready clips through automatic transcription, forced alignment, and phrase-aware segmentation, with a lightweight human review tool for quality control. Each professor is represented by a LoRA adapter fine-tuned on top of the VoxCPM 2 neural text-to-speech model, preserving speaking style while keeping training efficient on a single GPU. For deployment, we serve Qwen3-based automatic speech recognition through vLLM and stream synthesized speech from a customized nano-vllm runtime over WebSockets, supporting dynamic loading and swapping of per-professor LoRA weights without restarting the server. A React client lets users select a professor and hold a real-time voice session. This report describes the system design and implementation; quantitative evaluation is left to future work.

Contents

1 Introduction **2**
- 1.1 Goals . . . 2
- 1.2 Contributions . . . 2
- 1.3 Report organization . . . 2

2 Related Work **2**

3 System Overview **3**
- 3.1 Data Preprocessing . . . 3
- 3.2 Training . . . 4
- 3.3 Deployment . . . 4
- 3.4 Web Client . . . 4

4 Implementation **5**
- 4.1 Preprocessing Pipeline . . . 5
- 4.2 LoRA Fine-Tuning . . . 5
- 4.3 `nanovllm-voxcpm` Runtime . . . 6
- 4.4 Deployment Application . . . 6
- 4.5 Real-Time Voice Path (ASR + TTS) . . . 6

5 Discussion **6**

6 Conclusion **7**

1 Introduction

University teaching increasingly blends in-person lectures with asynchronous and interactive online formats. A *digital twin* of an instructor's voice could extend office hours, support accessibility, and deliver consistent explanations in the professor's own speaking style. Building such a twin from scratch requires not only a capable speech synthesis model, but also clean training data, efficient per-speaker adaptation, and a low-latency serving stack suitable for interactive use.

This project implements a complete pipeline—from scraped lecture audio to a deployable, multi-professor voice interface. We target **real-time voice-to-voice interaction**: the user speaks into a client application; the server transcribes incoming audio and synthesizes outgoing speech with the selected professor's voice. The deployed stack focuses on **automatic speech recognition (ASR)** and **text-to-speech (TTS)** inference; the client supplies or forwards the text that drives synthesis, so the server does not host a separate large language model for open-ended dialogue.

1.1 Goals

- Automate creation of 10–30 s training clips from full lectures with minimal manual editing.
- Fine-tune lightweight LoRA adapters for *multiple* professors on a shared VoxCPM 2 base model.
- Deploy ASR and TTS on one GPU with full-duplex WebSocket streaming and hot-swappable LoRA weights.
- Provide a web frontend to choose a professor and start a voice call.

1.2 Contributions

- A preprocessing pipeline combining Qwen3 ASR, forced alignment, and word-boundary-aware partitioning, plus an internal review tool.
- A multi-speaker dataset derived from ASU professor lectures, published on Kaggle.
- Per-professor LoRA fine-tuning of VoxCPM 2 with a reproducible training configuration.
- A modified `nanovllm-voxcpm` deployment with WebSocket streaming, dynamic `/loras` loading, and Prometheus/Grafana monitoring.

1.3 Report organization

Section 2 summarizes related work. Section 3 gives a system-level overview of preprocessing, training, deployment, and the client. Section 4 discusses implementation details. Section 5 covers limitations and ethics. Section 6 concludes.

2 Related Work

Neural text-to-speech and voice cloning. Modern TTS systems generate high-quality speech from text conditioning, often with speaker embeddings or fine-tuning to capture individual timbre and prosody. VoxCPM is a controllable speech model family suitable for voice-style transfer; we build on **VoxCPM 2** as a shared acoustic backbone and adapt it per professor.

Data preparation with ASR and alignment. Large-scale voice cloning benefits from accurately segmented utterances. Forced alignment links recognized words to timestamps in the audio waveform, enabling cuts at linguistically natural boundaries rather than fixed windows. We combine a strong ASR model (**Qwen3**) with a custom aligner and partitioning logic to produce consistent clips for training.

Parameter-efficient fine-tuning. Full fine-tuning of large generative models is costly and difficult to maintain for many speakers. **Low-Rank Adaptation (LoRA)** injects trainable low-rank matrices into selected layers while freezing most base weights, enabling one compact adapter per professor and fast switching at inference time.

High-throughput GPU serving. Serving frameworks such as **vLLM** batch requests and manage KV-cache memory efficiently for transformer inference. Our ASR endpoint uses vLLM; for TTS we extend a compact `nano-vllm`-style runtime with scheduling, block-based KV-cache management, and streaming audio output tailored to VoxCPM.

3 System Overview

Figure 1 shows the end-to-end pipeline: lecture ingestion, dataset creation, LoRA training per professor, and real-time serving with a web client.

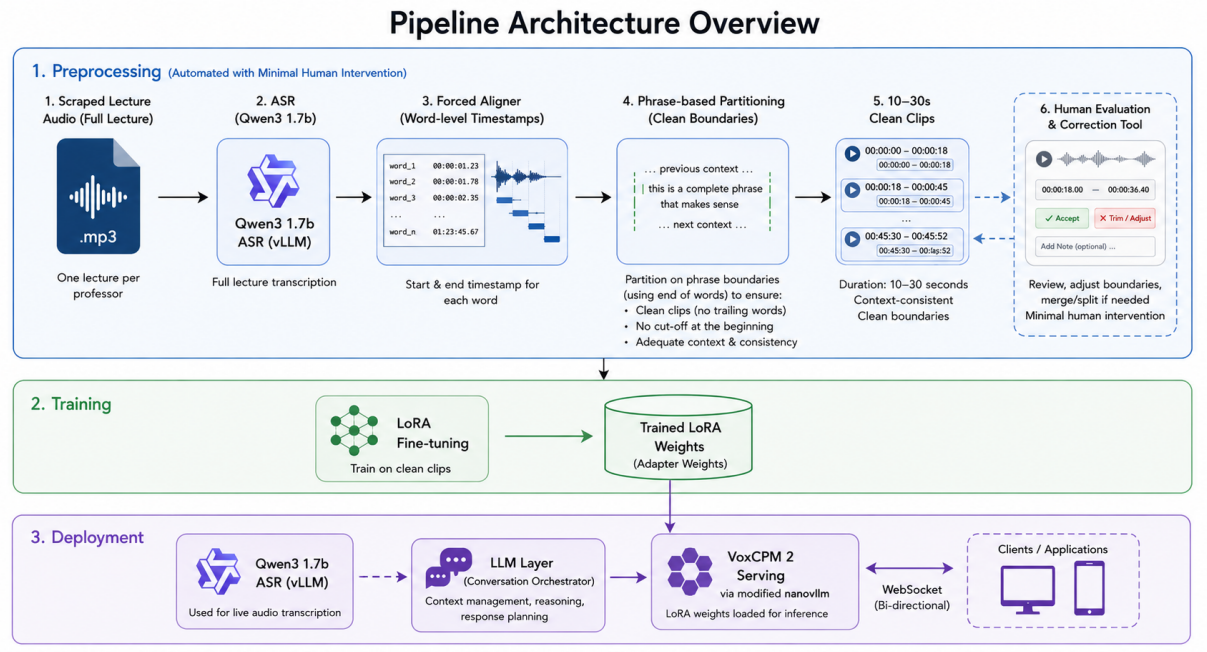

Figure 1: End-to-end pipeline from lecture audio to deployed professor voice twins.

3.1 Data Preprocessing

For each professor, we start from a scraped lecture recording (typically `.mp3`). The full file is transcribed with **Qwen3 1.7B ASR** and passed through a **forced aligner** to obtain word-level start and end times. A **partitioning algorithm** slices the audio at phrase and word boundaries so clips are roughly 10–30 seconds long, avoiding mid-word cuts while keeping segments suitable for TTS fine-tuning.

Clips pass through a small **human review tool** (Figure 2) for accept/reject and correction, reducing alignment errors before manifest generation. The same workflow applies to every professor in the cohort; processed segments are organized per speaker and published as the **ASU Professors Voice Cloning Dataset** on Kaggle.

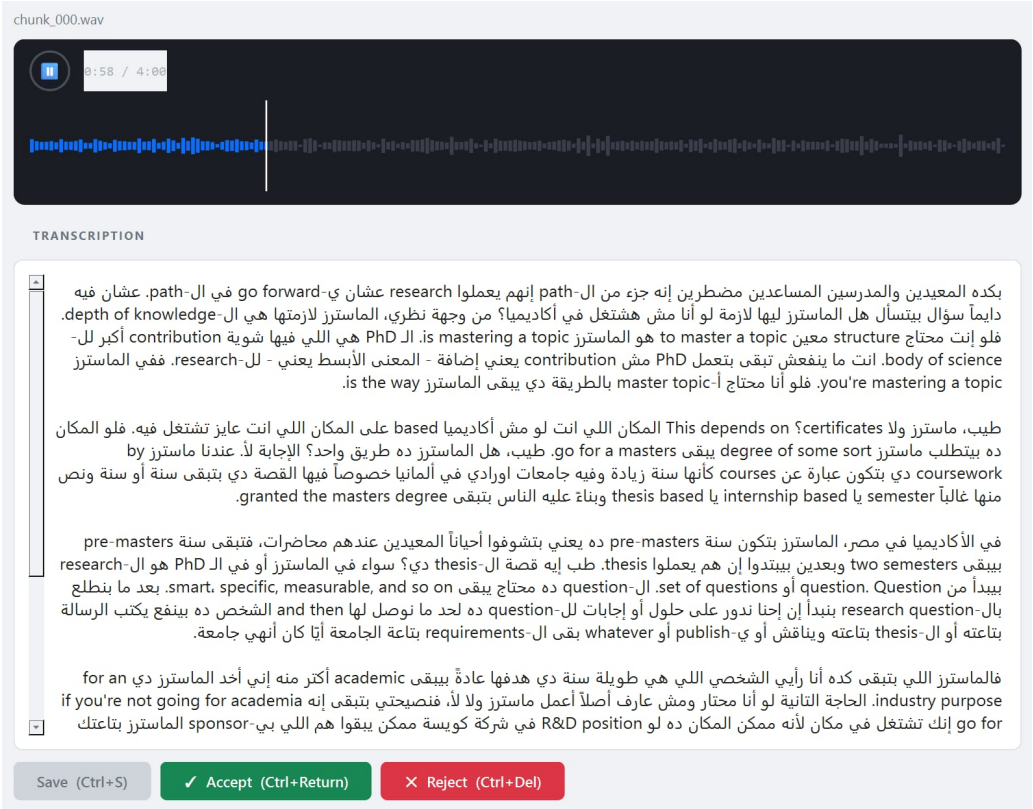

Figure 2: Internal tool for rapid human review of segmented clips.

3.2 Training

Training fine-tunes **LoRA adapters** on the frozen VoxCPM 2 base using JSONL manifests produced in preprocessing. Each professor has a dedicated adapter checkpoint (rank $r = 64$ on language-model and diffusion-transformer modules). Training runs on an NVIDIA A100 with gradient accumulation; checkpoints are saved periodically with a `latest` symlink for deployment convenience. Validation loss is logged when a held-out manifest is provided, but this report does not present a formal evaluation study.

3.3 Deployment

Both ASR and TTS run on a single **RTX A6000** host (Thunder Compute).

- **ASR:** Qwen3 served via vLLM for optimized batch transcription.
- **TTS:** VoxCPM 2 hosted in our `nanovllm-voxcpm` fork with WebSocket streaming and dynamic LoRA residency.

The asynchronous data path streams audio chunks from the client through ASR, forwards text into the TTS path, and returns synthesized audio chunks (Figure 3). Key HTTP/WebSocket endpoints include `WS /stream` (bidirectional audio), `POST /loras` (load or swap a professor's adapter without restart), and `GET /health` (orchestration health checks). **Prometheus** metrics and **Grafana** dashboards track latency, connection health, and GPU utilization.

3.4 Web Client

The **React + TypeScript** frontend lists available professors (avatars and metadata), establishes a WebSocket session to the deployment server, and captures/plays audio for a live voice

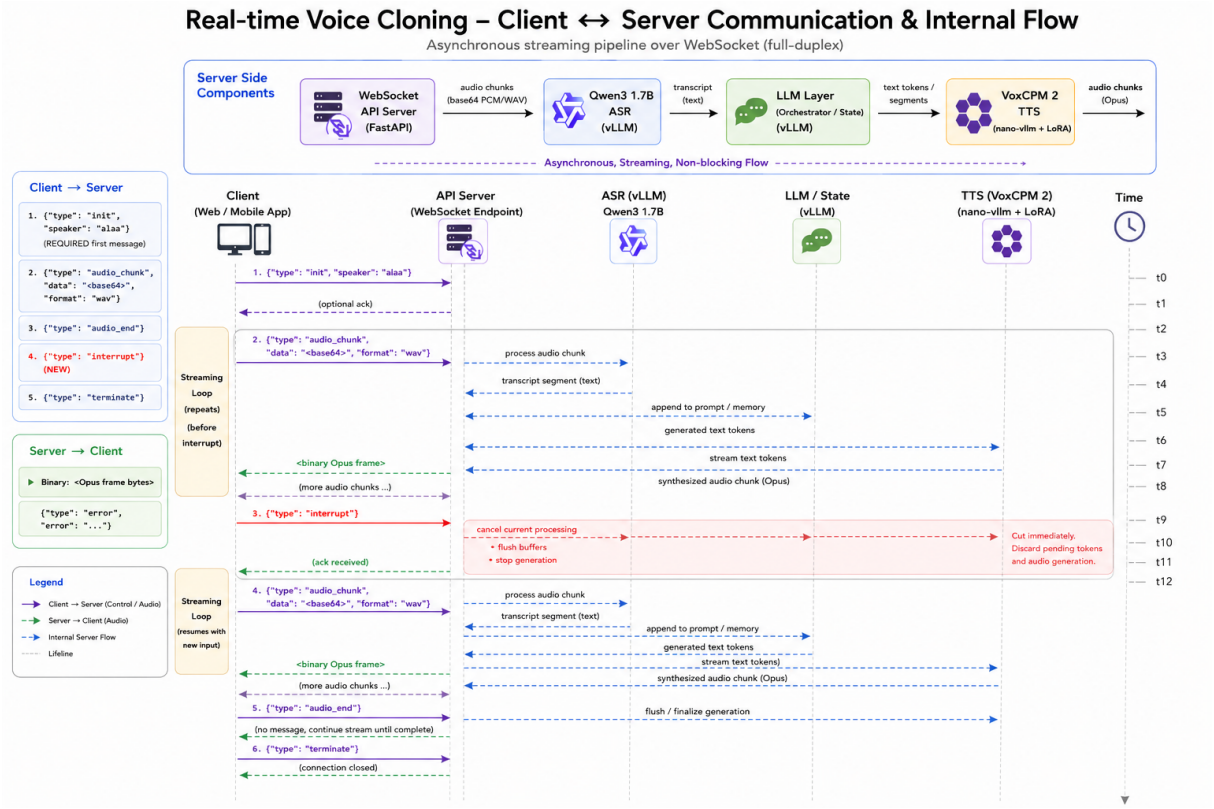

Figure 3: Client–server communication over WebSocket for streaming ASR and TTS.

call. Professor selection determines which LoRA adapter the backend loads via the `/loras` API before or during the session.

4 Implementation

This section highlights the main engineering components beyond the high-level overview.

4.1 Preprocessing Pipeline

The preprocessing package orchestrates transcription, alignment, segmentation, and manifest export (`Preprocessing/pipeline.py`). The transcriber wraps Qwen3 ASR output; the forced aligner refines token timings for reliable cut points. Partitioning prefers boundaries at the end of words and phrases so clips remain intelligible in isolation—important when the TTS model sees short utterances without surrounding lecture context.

The review tool (`Preprocessing/review_tool.py`) loads manifest rows, plays clips in the browser, and records reviewer decisions. This step scales across many professors: each speaker's manifest is reviewed independently, but the tooling and schema are shared.

4.2 LoRA Fine-Tuning

Training script `training/train_voxcpm_finetune.py` reads a YAML configuration specifying the pretrained VoxCPM 2 path, train/validation manifests, batch size, learning rate schedule, and LoRA targets. We enable LoRA on the language-model and DiT submodules (`enable_lm`, `enable_dit`) with rank and alpha 64, dropout 0, and optimize combined diffusion and stop losses. Audio is handled at 16 kHz for the encoder; the decoder outputs 48 kHz at inference.

TensorBoard logs training and validation metrics for debugging, though we do not report them here as formal results.

4.3 `nanovllm-voxcpm` Runtime

Our TTS server builds on a model-agnostic inference core:

- **Sequence** objects track per-request state, KV-cache block tables, and prefix tokens.
- A **Scheduler** batches sequences under memory limits.
- A **BlockManager** allocates shared KV blocks with prefix-cache hashing.
- **BaseModelRunner** executes the VoxCPM forward pass on GPU and returns waveform chunks via the audio VAE decoder.

The public `VoxCPM.from_pretrained` API spawns an **AsyncVoxCPMServerPool** (one server process per GPU) communicating over multiprocessing queues. VoxCPM-specific code maps text (and optional prompt latents) into model inputs and streams partial waveforms back to the client.

Design decisions are documented in architecture notes and ADRs inside the repository, including the rule that the GPU runner owns host–device transfers and explicit lifecycle rules for LoRA residency when swapping professors.

4.4 Deployment Application

The FastAPI application (`nanovllm-voxcpm/deployment/`) wires WebSocket handlers to a pipeline service that times each stage (ASR, TTS, encoding) and exports Prometheus histograms. Opus or raw audio codecs may be used depending on client negotiation; the pipeline records stage durations for observability.

Loading a professor twin at runtime:

```
POST /loras
{"name": "professor_id", "path": "/path/to/checkpoint"}
```

The server attaches the adapter to the running VoxCPM instance so multiple professors can be served from one base model without a full restart.

4.5 Real-Time Voice Path (ASR + TTS)

The deployed system implements a **real-time voice interface**: incoming audio is transcribed by the Qwen3 ASR service, and outgoing speech is synthesized by VoxCPM 2 with the active professor LoRA. The WebSocket endpoint focuses on low-latency audio I/O and model inference; text that drives TTS is provided through the session protocol from the client side rather than from an on-server dialogue model. This separation keeps GPU memory bounded and makes the stack easier to reason about for a deep-learning course project centered on speech models.

5 Discussion

Scalability across professors. The same preprocessing and training recipe applies to each new lecturer: scrape, segment, review, train a new LoRA, and register the checkpoint path with the deployment API. Operational cost grows with the number of adapters (disk, load time, and GPU memory for resident weights), so a production system might cache only hot speakers or use a tiered storage policy.

Data quality. Forced alignment errors and background noise in lecture halls still produce bad clips; human review mitigates but does not eliminate the burden as the roster grows. Future work could add automatic quality scoring (SNR, alignment confidence) before human review.

Compute and latency. Colocating ASR and TTS on one GPU simplifies deployment but creates contention under concurrent users. The nano-vllm scheduler and vLLM batching help, but we have not benchmarked tail latency in this report.

Ethics and responsible use. Cloning a real instructor's voice requires informed consent, clear disclosure when listeners interact with a synthetic voice, and policies against impersonation or misuse. The public dataset and demos should be distributed only with permission from the featured professors.

Scope limits. We do not model full conversational intelligence on the server; prosody and content of replies depend on whatever text the client supplies. Integrating a dialogue LLM, emotion control, or multilingual support would be natural extensions but are out of scope for the current system.

6 Conclusion

We built a multi-professor **voice digital twin** pipeline spanning automated lecture segmentation, LoRA fine-tuning of VoxCPM 2, and a WebSocket-based deployment that couples Qwen3 ASR with streaming TTS and hot-swappable adapters. The design prioritizes reuse: one base model, many lightweight professor-specific LoRAs, and shared tooling for data and operations.

Future directions include automated clip quality filtering, formal subjective and objective evaluation, multi-GPU scaling, and optional integration of a dialogue module while keeping the speech stack modular.